

INTERNPE INTERNSHIP/TRAINING PROGRAM

Python 2(NOTES CONTENT)



Chapter 6: Modules and Packages

Introduction to Modules and Packages: In the world of Python programming, modules and packages are tools that help organize and manage code in a structured and efficient manner. They allow you to break down your code into smaller, manageable units, promoting code reusability, readability, and collaboration.

Understanding Modules: A module is a single Python file that contains functions, classes, and variables. It acts as a container for related code. Modules help keep your codebase organized and make it easier to maintain and navigate.

Creating and Using Modules:

Creating a Module: Create a new `.py` file with a relevant name, such as `my_module.py`.

Defining Functions/Classes: Write your functions, classes, or variables within the module file.

Importing Modules: In another Python file, import the module using the `import` keyword.

Accessing Contents: Access the functions, classes, or variables using dot notation:

```
module_name.function_name().
```

```
le.py
def greet(name):
    print(f"Hello, {name}!")

# main.py
import my_module
my_module.greet("Alice") # Outputs: Hello, Alice!
```

Understanding Packages: A package is a directory that contains multiple modules. It helps group related modules together, making it easier to manage and structure code for larger projects.

Creating and Using Packages:

Creating a Package: Create a directory (folder) and give it a meaningful name, like `my_package`.

Module Files: Within the package directory, create `.py` files, each serving as a module.

Importing from Packages: Import modules from the package using dot notation.

```
# ├── __init__.py
# ├── module1.py
# └── module2.py

# main.py
from my_package import module1
module1.function_name()
```

Benefits of Using Modules and Packages:

Organization: Code can be logically divided into smaller units, improving readability and maintainability.

Reusability: Modules and packages can be reused across different projects, saving development time.

Collaboration: Large projects can be divided among team members, with different modules being developed simultaneously.

Encapsulation: Modules and packages help encapsulate functionality, reducing code interference.

Avoiding Name Conflicts: They prevent naming conflicts by keeping related code separate.

Third-Party Packages: Apart from built-in modules and your own packages, you can use third-party packages created by the Python community. These packages offer a wide range of functionalities, from data manipulation to web development.

Installing Third-Party Packages: You can use `pip`, the Python package manager, to install third-party packages.

```
pip install package_name
```

Using Third-Party Packages: Once installed, you can import and utilize the package's modules in your code.

Conclusion: Modules and packages are foundational concepts in Python programming. They provide a structured approach to writing and organizing code, making it more efficient, maintainable, and adaptable to various programming tasks. By effectively utilizing modules and packages, you can streamline your development process and create more organized and readable Python projects.

Chapter 7: Reading from Files

Introduction to Reading Files: Imagine your computer's memory is like a whiteboard. You write data on it, but when you turn off the computer, the data disappears. Files are like notebooks where you can write and store information permanently. In Python, you can read data from files to use in your programs.

Opening Files: Before reading from a file, you need to open it. Think of it like opening a book before reading. You use the `open()` function:

```
python
file = open("myfile.txt", "r") # "r" means read mode
```

Reading Methods: Once the file is open, you can use different methods to read its content:

`read()` Method: Reads the entire content of the file as a string.

python

```
content = file.read()
print(content)
```

`readline()` Method: Reads one line at a time.

python

```
line = file.readline()
print(line)
```

`readlines()` Method: Reads all lines and returns a list.

python

```
lines = file.readlines()
for line in lines:
    print(line)
```

Closing the File: Just as you close a book after reading, you need to close the file when you're done using it:

python

```
file.close()
```

Using `with` Statements: A better way to handle files is by using `with` statements. It automatically closes the file when you're done:

python

```
with open("myfile.txt", "r") as file:
    content = file.read()
    print(content)
# File is automatically closed here
```

Working with Paths: If your file is in a different folder, you need to specify the path to the file.

Think of it like telling someone where to find a specific notebook:

python

```
with open("folder/myfile.txt", "r") as file:  
    content = file.read()  
    print(content)
```

Conclusion: Reading from files is like extracting information from a notebook. Python provides simple tools to open, read, and work with file content. Just remember to close the file or use the `with` statement to manage files efficiently and prevent memory leaks.

Chapter 8: Exception Handling

Introduction to Exception Handling: Imagine you're baking a cake and the recipe asks for eggs.

What if you don't have eggs? You wouldn't want your entire cake-making process to stop.

Exception handling in Python is like having a backup plan for such situations. It helps your program gracefully handle errors and continue running.

What are Exceptions? In programming, exceptions are unexpected errors that can occur when your code is executed. For example, dividing a number by zero or trying to access a file that doesn't exist can cause exceptions.

Using `try` and `except`: To handle exceptions, you use the `try` and `except` blocks. It's like telling your program, "Try doing this, and if something goes wrong, do something else."

python

```
try:  
    num = int(input("Enter a number: "))  
    result = 10 / num  
    print(result)
```

```
except ZeroDivisionError:
    print("Oops! You can't divide by zero.")
except ValueError:
    print("Oops! That's not a valid number.")
```

Handling Multiple Exceptions: You can handle different types of exceptions using separate `except` blocks.

python

```
try:
    # Some code that might cause exceptions
except ZeroDivisionError:
    # Handle division by zero error
except ValueError:
    # Handle invalid value error
except AnotherException:
    # Handle another specific exception
```

Using `else` and `finally`:

- `else`: You can use an `else` block to specify code that should run when no exceptions occur.
- `finally`: The `finally` block contains code that always runs, regardless of whether an exception occurred or not. It's like cleaning up after yourself.

python

```
try:
    # Some code that might cause exceptions
except MyException:
    # Handle the exception
else:
    # Code to run if no exceptions occurred
finally:
    # Code that always runs, whether an exception occurred or not
```

Raising Exceptions: Sometimes you want to raise an exception intentionally to indicate that something unexpected happened. You can do this using the `raise` statement.

python

```
def check_age(age):  
    if age < 0:  
        raise ValueError("Age cannot be negative")
```

Conclusion: Exception handling is like having a safety net for your program. It allows you to gracefully handle unexpected errors, preventing your program from crashing. By using `try`, `except`, and related constructs, you can ensure that your program keeps running smoothly even in the face of unexpected challenges.

Chapter 9: Object-Oriented Programming (OOP)

Introduction to Object-Oriented Programming (OOP): Think of a car – it has properties like color and speed, and it can perform actions like starting and stopping. Object-Oriented Programming (OOP) is a way to model real-world entities in your code by creating "objects" that have both properties (attributes) and behaviors (methods). This approach makes your code more organized and easier to manage.

Objects, Classes, and Instances:

- **Objects:** Objects are like instances of a class. Just as you can have different cars of the same model, objects are specific instances of a general concept.
- **Classes:** A class is like a blueprint that defines how objects of that class will behave and what attributes they will have.

Defining a Class: Imagine a blueprint for a car. It specifies attributes like color and speed, and methods like start and stop.

python

```
class Car:
    def __init__(self, color, speed):
        self.color = color
        self.speed = speed

    def start(self):
        print("Car started")

    def stop(self):
        print("Car stopped")
```

Creating Objects (Instances): Using the class blueprint, you can create actual car objects with specific colors and speeds.

python

```
my_car = Car("red", 100)
your_car = Car("blue", 120)
```

Accessing Attributes and Methods: You can access attributes and methods of objects using dot notation.

python

```
print(my_car.color) # Outputs: red
my_car.start() # Outputs: Car started
```

Inheritance: Inheritance is like a family tree. You can create a new class that inherits properties and methods from an existing class.

python

```
class ElectricCar(Car):
```



```
def charge(self):  
    print("Car is charging")
```

Encapsulation: Encapsulation means bundling data (attributes) and methods that operate on the data into a single unit (object or class). It hides the complexity and allows you to control access to data.

Polymorphism: Polymorphism is like using the same interface (method name) for different types of objects. It allows you to write code that can work with various classes in a consistent way.

Conclusion: Object-Oriented Programming is a way to make your code mimic real-world entities. By defining classes, creating objects, and using principles like inheritance and encapsulation, you can build more organized, modular, and maintainable code. OOP helps you think about your program's structure in terms of objects and their interactions, leading to better code design and development.

Chapter 10: Built-in Functions and Libraries

Introduction to Built-in Functions: Imagine having a toolbox full of tools you can use instantly.

Python comes with a collection of ready-to-use tools called "built-in functions." These functions perform common tasks, saving you time and effort by providing shortcuts for various operations.

Using Built-in Functions: To use a built-in function, you simply call it with the required inputs (arguments). Think of it as telling the tool what you want it to do.

python

```
result = len("Python") # Calculates the length of the string "Python"
print(result) # Outputs: 6
```

Common Built-in Functions: Python offers a variety of built-in functions for different tasks. Here are a few examples:

- `len()`: Calculates the length of a string, list, or other sequence.
- `type()`: Determines the data type of a value.
- `int()`, `float()`, `str()`, `bool()`: Convert values to specific data types.
- `print()`: Outputs data to the console.
- `input()`: Accepts user input from the console.
- `range()`: Generates a sequence of numbers.
- `sum()`, `max()`, `min()`: Perform calculations on sequences of numbers.

Using Libraries: Libraries are collections of code that extend Python's capabilities beyond built-in functions. Libraries contain functions and modules that others have written, saving you from reinventing the wheel.

Importing Libraries: To use a library, you need to import it into your code. Think of it as opening a toolkit that contains specialized tools.

python

```
import math # Importing the math library
```

Using Functions from Libraries: Once you've imported a library, you can use its functions by calling them using dot notation.

python

```
result = math.sqrt(25) # Using the square root function from the math library
```

```
print(result) # Outputs: 5.0
```

Common Libraries: Python has a vast library ecosystem for various purposes. Here are a few commonly used ones:

- `math`: Provides mathematical functions like square root and trigonometry.
- `random`: Generates random numbers and choices.
- `datetime`: Works with dates and times.
- `os`: Interacts with the operating system (e.g., file operations).
- `requests`: Makes HTTP requests for web interactions.
- `numpy`, `pandas`: Used for data manipulation and analysis.
- `matplotlib`, `seaborn`: Used for data visualization.

Conclusion: Built-in functions and libraries are like shortcuts and toolkits that make your coding life easier. They provide you with pre-made solutions for common tasks and extend Python's capabilities. By using these tools, you can write more efficient and powerful code without having to start from scratch every time.